THE MINISTRY OF SCIENCE AND HIGHER EDUCATION
OF THE RUSSIAN FEDERATION

ITMO University
(ITMO)

Faculty of Control Systems and Robotics

Task 2
for the subject
"Simulation of Robotic Systems"

Student:
Group No. R4137c *Mughal, Umer Ahmed Baig*
ITMO ID. 472268

Tutor:
Professor *Borisov, Ivan*

Assistant *Rakshin, Egor*

Saint Petersburg 2024

CONTENTS

0.1 Objective 1
0.2 Tasks 1

1 RR MECHANISM WITHOUT TENDONS 2
1.1 XML Model of the RR Mechanism without Tendons 2

2 INTEGRATING TENDONS TO THE RR MECHANISM 5
2.1 XML Model of the RR Mechanism with Tendons 5

3 PYTHON SCRIPT FOR EXECUTING THE XML MODEL 7

4 CONCLUSION 14

TASK DESCRIPTION

0.1 Objective:

To perform simulation modeling of passive RR mechanism of open kinematics with two elastic cables.

0.2 Tasks:

1. Study examples of using cables in other XML files.
2. Compose an RR mechanism without tendons.
3. Add two tendons thrown over pulleys.
4. Make a script that runs the XML model.
5. Make graphs and write a report with explanations.

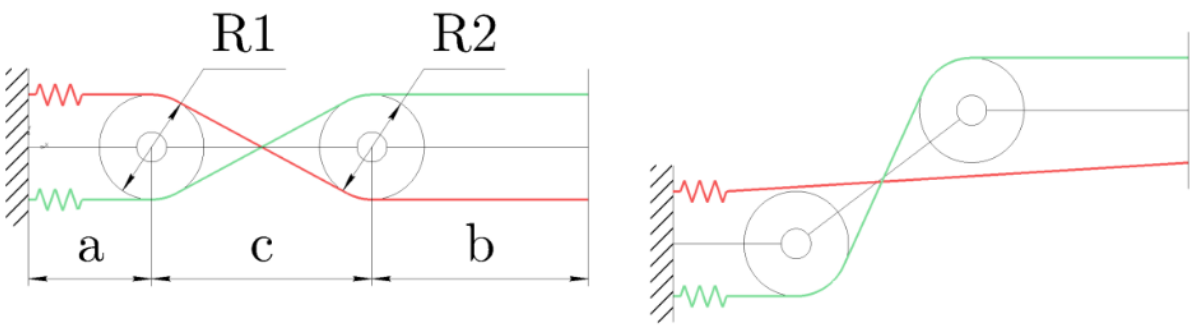

Figure 1: - RR mechanism of open kinematics with two elastic cables.

The initial data for completing the task are presented in Table 1.

Table 1

a, m	b, m	c, m	R1, m	R2, m
0.045	0.039	0.055	0.01	0.01

Given values of the parameters according to the task.

1 RR MECHANISM WITHOUT TENDONS

Using the provided parameters, the RR mechanism will initially be created without any tendons (cables). This allows for the development of a baseline model that focuses on the two rotational joints and their motion. The RR mechanism consists of rotational joints utilizing pulleys, which will be simulated in the MuJoCo environment.

1.1 XML Model of the RR Mechanism without Tendons

To create the RR mechanism model, key XML commands such as **<geom>** for defining geometric bodies and **<joint>** for creating joints are utilized. By analyzing examples and understanding MuJoCo's commands, the RR mechanism is first modeled without tendons, as shown in Figure 2.

Below is a script that creates an XML model of RR mechanism without tendons.

```
<mujoco model="RR_Mechanism_Model_472268">

      <statistic center="0.065 0 0.285" extent="0.225" />

<visual>
      <headlight diffuse="0.6 0.6 0.6" ambient="0.25 0.25 0.25"/>
      <rgba haze="0.05 0.1 0.15 1" />
      <global azimuth="85" elevation="-10" />
</visual>

<asset>
      <texture type="skybox" builtin="gradient" rgb1="0.3 0.5 0.7" rgb2="0.1 0.2
       0.3" width="512" height="512" />
      <texture type="2d" name="groundplane" builtin="checker" mark="edge"
       rgb1="0.5 0.6 0.7" rgb2="0.005 0.005 0.005" markrgb="0.8 0.8 0.8"
       width="300" height="300" />
      <material name="groundplane" texture="groundplane" texuniform="true"
       texrepeat="10 10" reflectance="0.2" />
</asset>

<default>
      <joint springdamper="1 100" axis="0 1 0" limited="true" range="-90 90" />
      <geom type="capsule" size=".0045" />
      <site rgba="0.5 0.5 0.5 1" />
      <default class="visual_cylinder">
      <geom type="cylinder" fromto="0 .01 0 0 -.01 0" size=".01"
       rgba=".5 .0 .5 .5" />
 </default>
</default>
```

```
<worldbody>
      <light pos="0 0 1.5" />
      <light pos="0 0 3" dir="0 0 -1" directional="false" />
      <geom name="floor" size="0 0 .125" type="plane" material="groundplane"
       conaffinity="15" condim="3" />

<body pos="0 0 .3">
      <geom name='left_wall' type="box" size="0.003 0.04 0.02"
       rgba="0.35 0.35 0.35 1" pos=" -0.003 0 0" />
      <geom fromto="0 0 0 .045 0 0" contype="0" conaffinity="0" density="5000" />
      <site name="s1" pos="0 0 0.015" size=".004" rgba="0 0 0 1" />
      <site name="s2" pos="0 0 -0.015" size=".004" rgba="0 0 0 1" />
 <body pos=".045 0 0">
      <joint name="one" axis="0 1 0" />
      <geom fromto="0 0 0 .039 0 0" density="5000" />
      <geom name="Pulley1" type="cylinder" fromto="0 .01 0 0 -.01 0" size=".01"
       rgba=".5 .0 .5 .5" />
      <site name="s3" pos="0 0 0.012" size=".004" rgba="0 0 0 1" />
      <site name="s4" pos="0 0 -0.012" size=".004" rgba="0 0 0 1" />

 <site name="c1" pos=".02 0 0" size=".004" rgba="0.5 0.5 0.5 1" />

 <body pos=".039 0 0">
      <joint name="two" />
      <geom name="Pulley2" type="cylinder" fromto="0 .01 0 0 -.01 0" size=".01"
       rgba=".5 .0 .5 .5" />
      <site name="s5" pos="0 0 -0.012" size=".004" rgba="0 0 0 1" />
      <site name="s6" pos="0 0 0.012" size=".004" rgba="0 0 0 1" />
      <geom fromto="0 0 0 .055 0 0" density="5000" />
      <site name="s7" pos=".055 0 0.015" size=".004" rgba="0 0 0 1" />
      <site name="s8" pos=".055 0 -0.015" size=".004" rgba="0 0 0 1" />
<body>
       <geom name='right_wall' type="box" size="0.001 0.02 0.0175"
        rgba="0.35 0.35 0.35 1" pos=" 0.056 0 0" />
 </body>
 </body>
 </body>
 </body>
</worldbody>

</mujoco>
```

The XML model structures the RR mechanism by defining statistics, visual settings, assets, defaults, and primary components such as geometries and joints. The visual settings enhance rendering through lighting adjustments, depth effects, and camera configuration. Assets include a gradient skybox texture for the background and ground-plane texture with tiling and reflectance for visual appeal. Default parameters streamline the model, specifying joint properties like spring-damper settings, range limits, and axis orientation, as well as geometric and site defaults. The

modeling process uses key MuJoCo commands, resulting in an efficient XML representation of the RR mechanism without tendons.

Figure 2 shows the XML model of the RR mechanism without tendons.

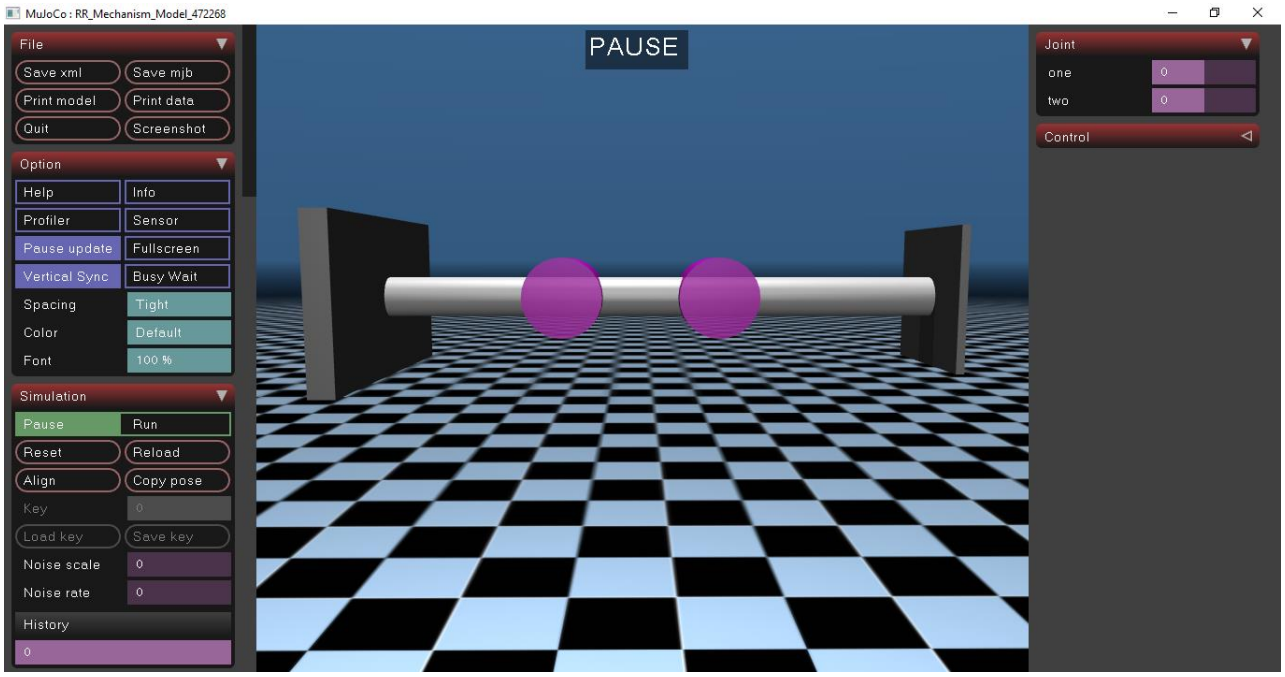

Figure 2: XML Model of RR Mechanism without Tendons.

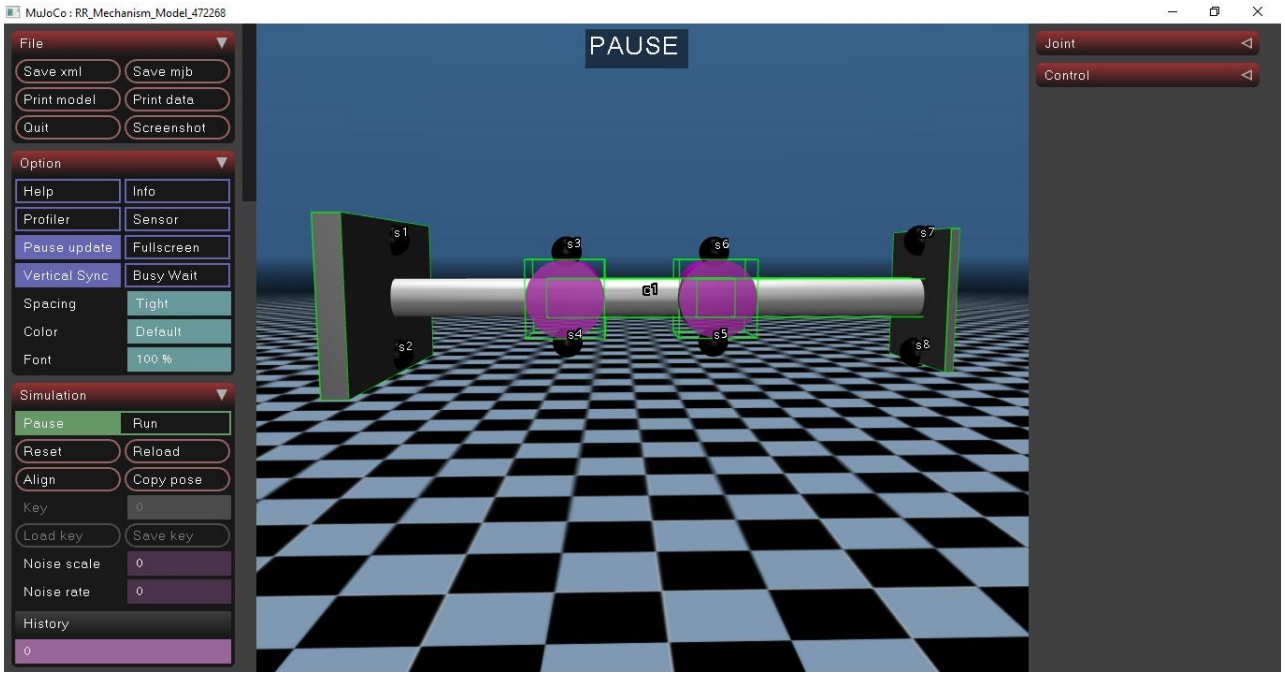

Figure 3: XML Model of RR Mechanism with Sites.

2 INTEGRATING TENDONS TO RR MECHANISM

After the basic RR mechanism is created, the next step is to incorporate the two elastic tendons. These tendons will be thrown over pulleys attached to the mechanism. The tendons will add a spring-like force to the system, creating tension that interacts with the RR mechanism's movement, simulating an elastic response in the joints and structure.

The RR mechanism consists of two rotational joints ("one" and "two"), with the parameters a=0.045 m, b=0.039 m, c=0.055, R1=0.01m and R2=0.01m defining the dimensions of the system. Elastic tendons (cables) are modeled using the <tendon> element with spatial stiffness, accurately simulating the elastic forces applied to the joints. Pulleys guide the tendons along the RR mechanism.

2.1 XML Model of the RR Mechanism with Tendons

To add cables, the code below was compiled to create an XML model of RR mechanism with tendons:

```
<tendon>
    <spatial stiffness="10" rgba="1 0 0 1">
        <site site="s1" />
        <geom geom="Pulley1" sidesite="s3" />
        <site site="c1" />
        <geom geom="Pulley2" sidesite="s5" />
        <site site="s8" />
    </spatial>
    <spatial stiffness="10" rgba="0 1 0 1">
        <site site="s2" />
        <geom geom="Pulley1" sidesite="s4" />
        <site site="c1" />
        <geom geom="Pulley2" sidesite="s6" />
        <site site="s7" />
    </spatial>
</tendon>
```

Main commands used in the code:

– <spatial stiffness="10"/>: Set the stiffness of the spring and its visual characteristics.

– <site site="s1"/>: Set which 2 points need to be connected.

– <geom geom="g1"sidesite = "side1"/>: Defines the body through which it is possible to pass and specifies the side from which it is rounded.

Figure 4 illustrates the XML model of the RR mechanism with tendons.

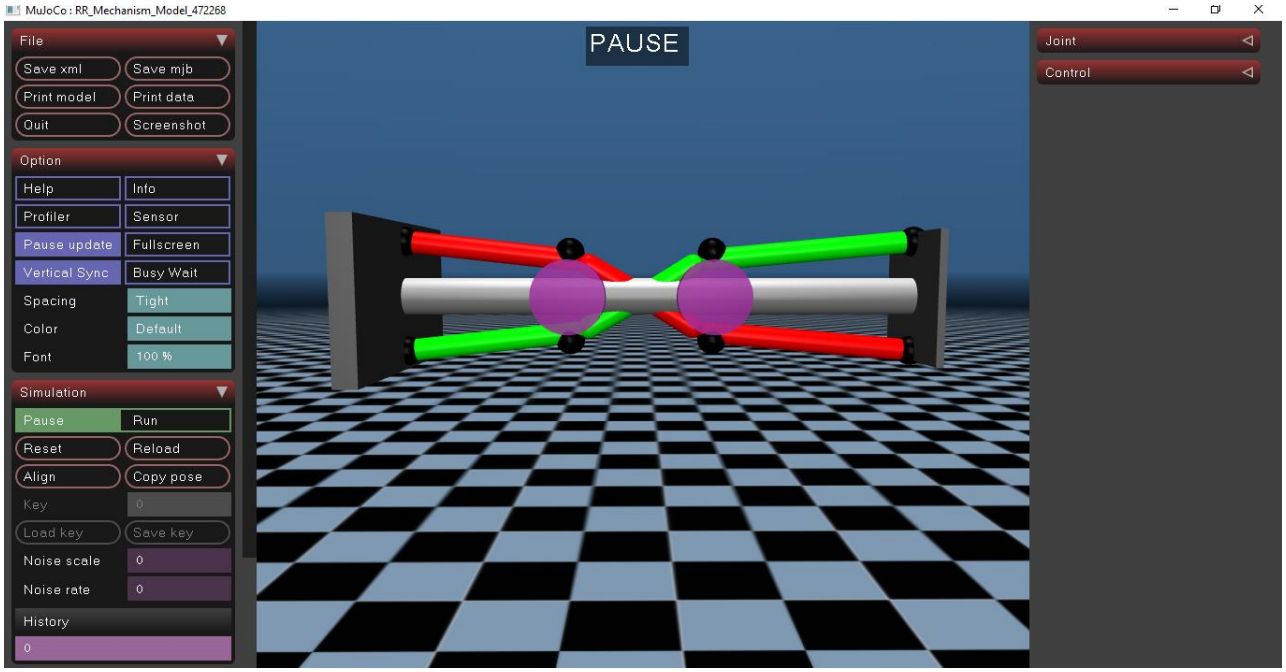

Figure 4: XML Model with Tendons

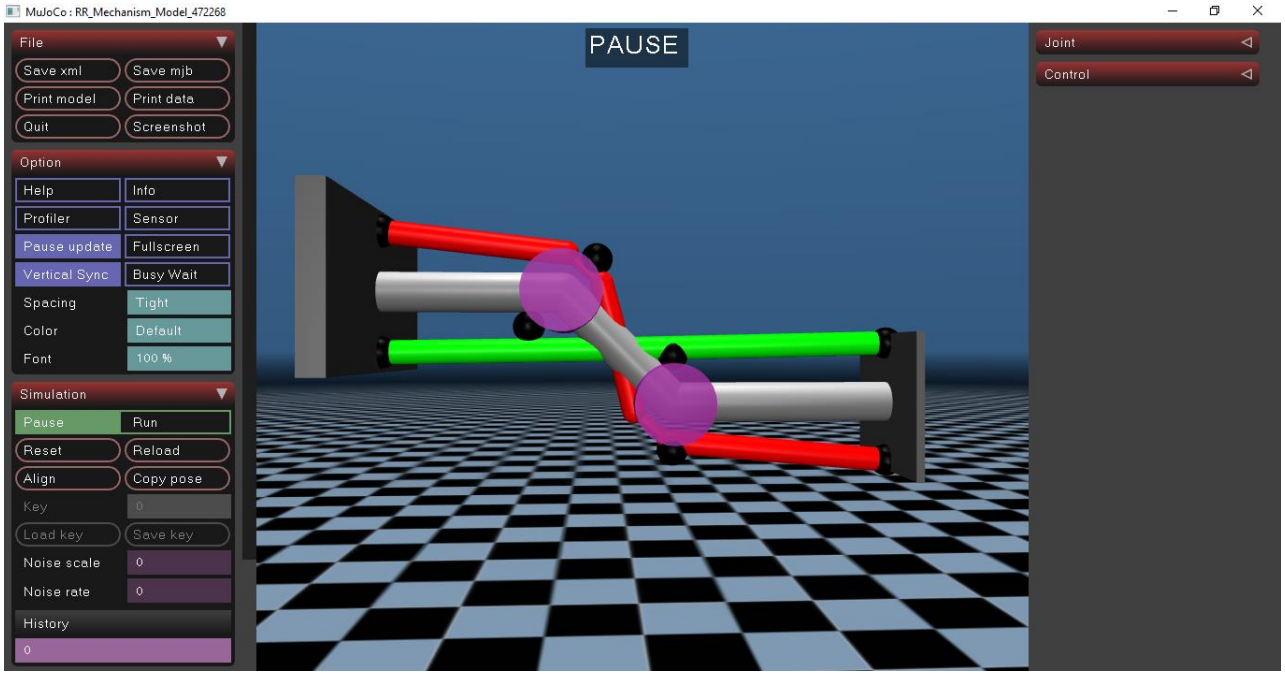

Figure 5: RR Mechanism of XML Model with Tendons

3 PYTHON SCRIPT FOR EXECUTING THE XML MODEL

A script is created to load the XML model into MuJoCo and run the simulation, enabling real-time visualization and analysis of the RR mechanism's behavior under the influence of the tendons. The script manages the simulation parameters, initial conditions, and time steps for the model. Below is the Python program that executes the XML model and collects data about the coordinates of the joints in the system at different moments in time.

```
import time
import mujoco
import mujoco.viewer
import matplotlib.pyplot as plt

# Load MuJoCo model and data
m = mujoco.MjModel.from_xml_path('C:\\Users\\ADMIN\\Desktop\\Task2.xml')
d = mujoco.MjData(m)

# Initialize lists for joint values
x_values = []
y_values = []

# Launch passive viewer
with mujoco.viewer.launch_passive(m, d) as viewer:
    start = time.time()

    # Run the simulation for 5 seconds
    while viewer.is_running() and time.time() - start < 5:
        step_start = time.time()

        # Append joint positions to the respective lists
        y_values.append(float(d.qpos[m.jnt_qposadr[mujoco.mj_name2id(m,
        mujoco.mjtObj.mjOBJ_JOINT, 'one')]]))
        x_values.append(float(d.qpos[m.jnt_qposadr[mujoco.mj_name2id(m,
        mujoco.mjtObj.mjOBJ_JOINT, 'two')]]))

        # Step simulation
        mujoco.mj_step(m, d)

        # Update viewer settings
        with viewer.lock():
          viewer.opt.flags[mujoco.mjtVisFlag.mjVIS_CONTACTPOINT] = int(d.time % 2)

        # Sync viewer
        viewer.sync()

        # Ensure correct simulation timing
        time_until_next_step = m.opt.timestep - (time.time() - step_start)
        if time_until_next_step > 0:
```

```
            time.sleep(time_until_next_step)

# Plot the first joint's position
plt.figure()
plt.plot(y_values, label="J1")
plt.ylabel("J1")
plt.title("Joint One Position Over Time")
plt.grid(True)
plt.legend()
plt.show()

# Plot the second joint's position
plt.figure()
plt.plot(x_values, label="J2")
plt.ylabel("J2")
plt.title("Joint Two Position Over Time")
plt.grid(True)
plt.legend()
plt.show()
```

The script demonstrates the use of the MuJoCo to simulate and visualize the behavior of the RR mechanism defined in the XML model file (**Task2.xml**). The simulation collects and plots the positions of two joints (J1 and J2) over a specified period of time.

- **Loading the Model:**
  - The XML model file located at **"C:\\Users\\ADMIN\\Desktop\\Task2.xml"** is loaded into the MuJoCo environment using the **'MjModel'** and **'MjData'** classes.
  - **'MjModel'** contains the static information about the model, and **'MjData'** contains the dynamic states during simulation (e.g., joint positions, velocities).
- **Launching the Viewer:**
  - A passive **'MuJoCo viewer'** is launched to visually monitor the simulation in real-time. The viewer also allows toggling specific visualization flags (e.g., showing contact points).
- **Simulation Process:** The simulation runs for specified time, during which:
  - The position of 'Joint 1' (J1) and 'Joint 2' (J2) are extracted at every simulation step using the **'qpos'** property, which holds the generalized positions of all joints.
  - These values are stored in separate lists (**'x_values'** for J2 and **'y_values'** for J1) for plotting later.
  - The MuJoCo simulation advances using the **'mj_step()'** function, which computes and updates the model state for the next time step.

- **Viewer Settings:**
  - The viewer's visualization options are dynamically updated to toggle contact point visibility **'mjVIS_CONTACTPOINT'** every 2 seconds, providing additional visual cues in the simulation.

- **Plotting Joint Positions:**
  - After the simulation, the joint positions over time are visualized using Matplotlib:
    - The first plot shows the Joint 1's position (J1) over time.
    - The second plot shows the Joint 2's position (J2) time.
  - These plots include labels, titles, grids, and legends for clarity.

When executing this code, 2 graphs were obtained based on the specified simulation time.

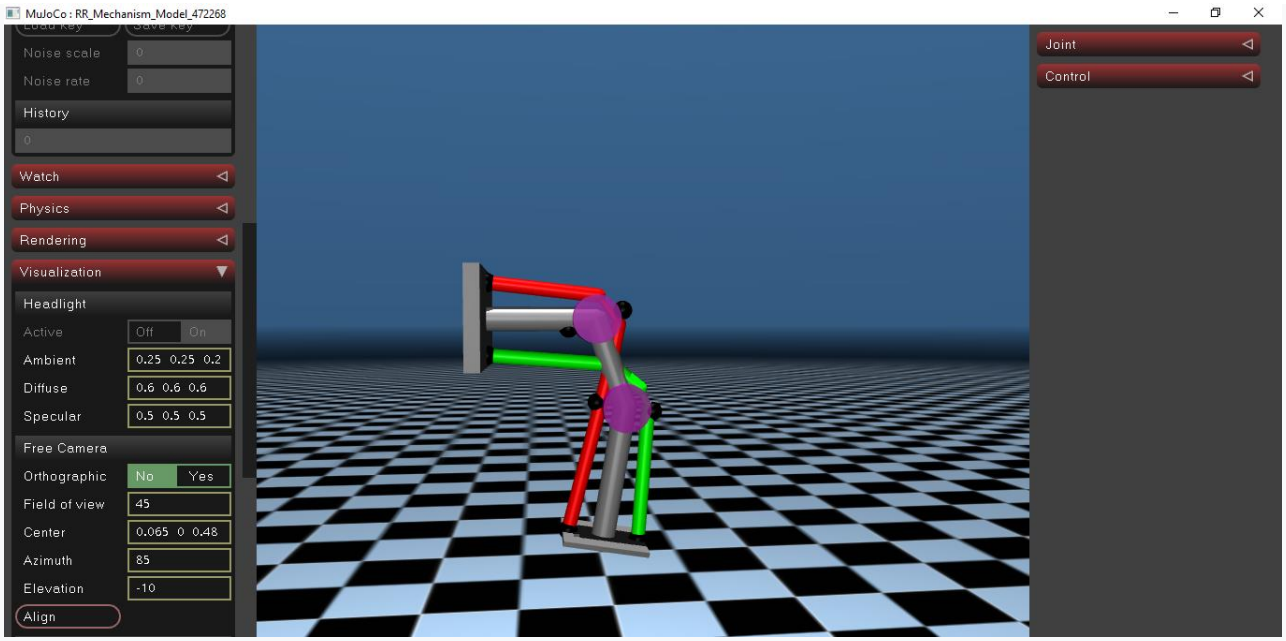

Figure 6: XML Model of RR Mechanism during the Simulation Phase.

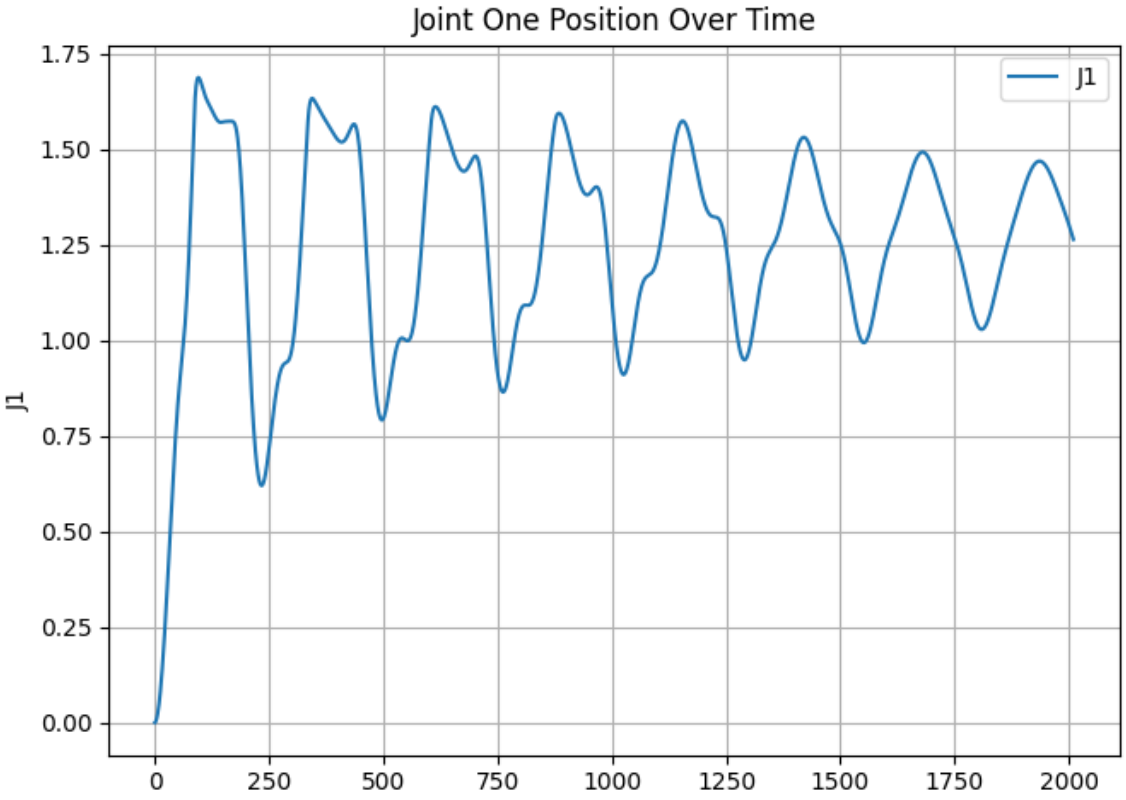

Figure 7: Value of the generalized coordinate J1 when the simulation runs for 5Sec.

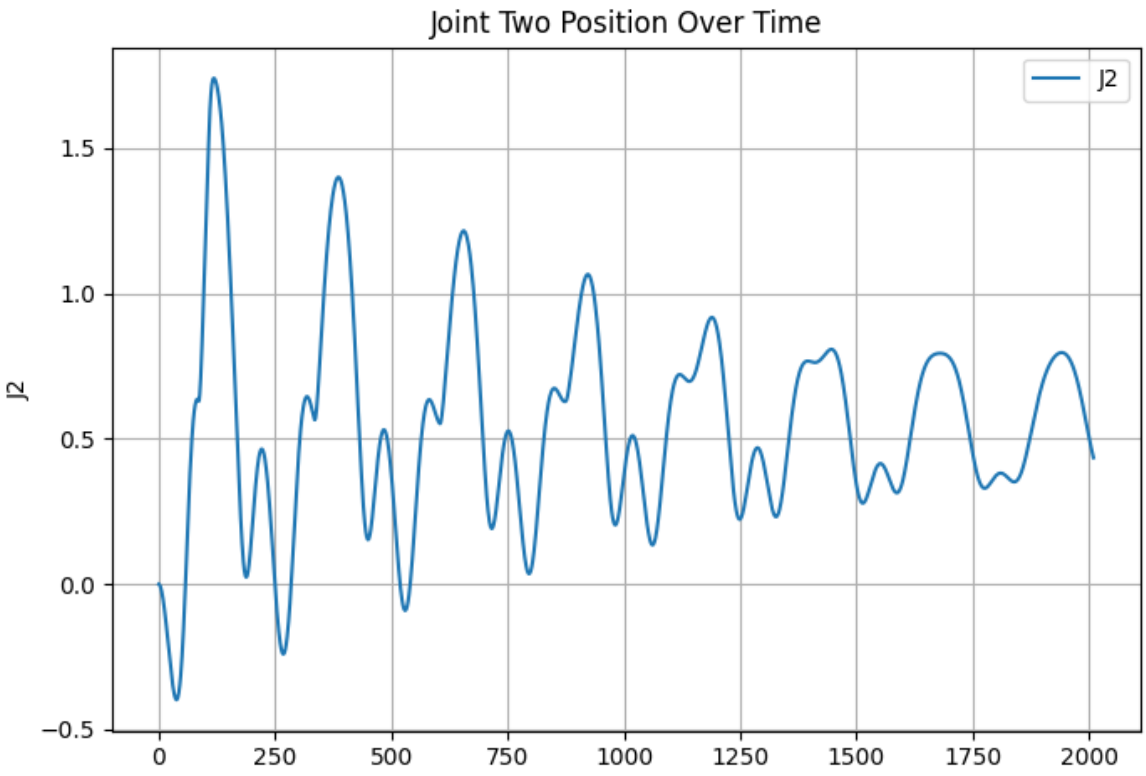

Figure 8: Value of the generalized coordinate J2 when the simulation runs for 5Sec.

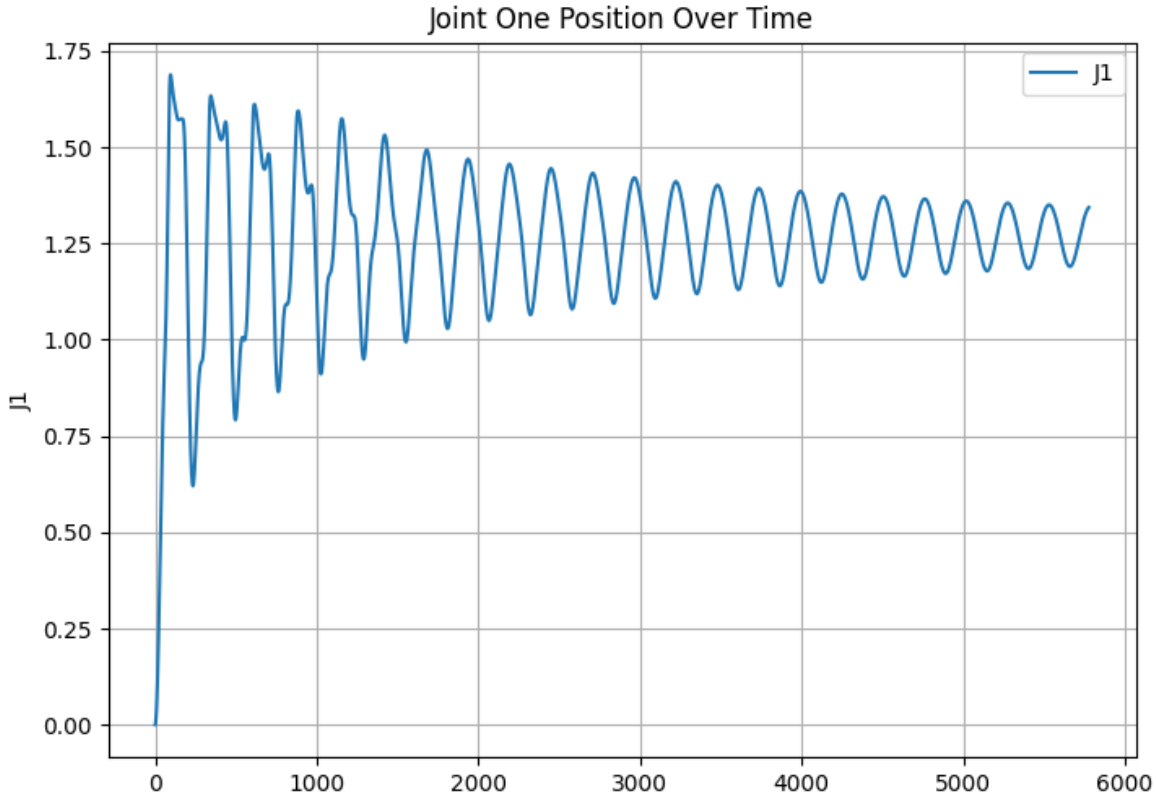

Figure 9: Value of the generalized coordinate J1 when the simulation time increased to 15Sec.

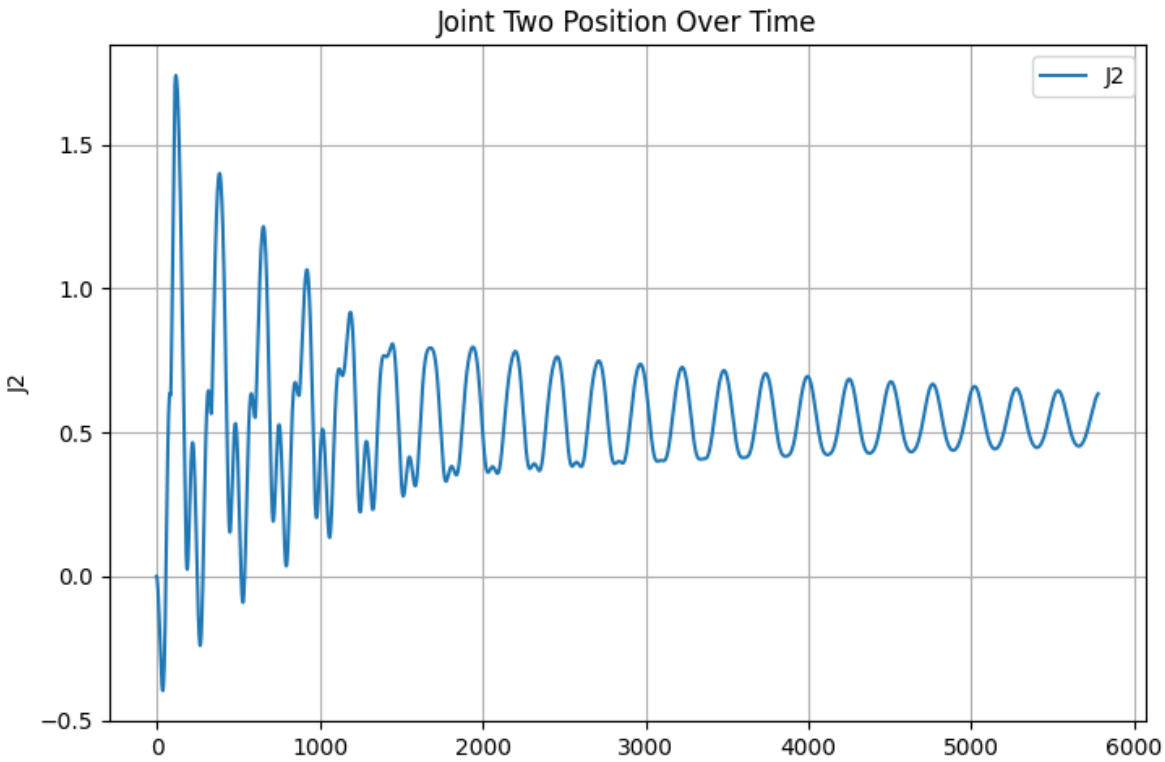

Figure 10: Value of the generalized coordinate J2 when the simulation time increased to 15Sec.

The graphs below present a comparison of the positions of Joint J1 and Joint J2 at simulation steps of 5 seconds and 15 seconds.

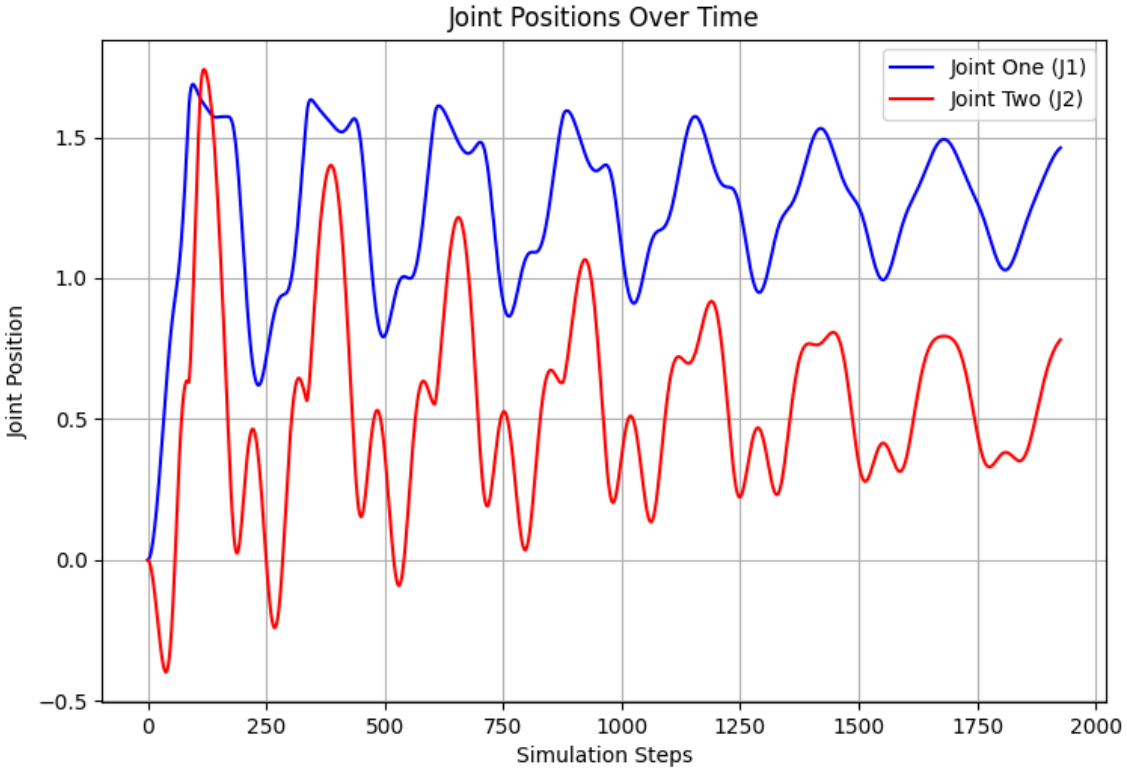

Figure 11: Value of the generalized coordinate J1 and J2 when the simulation time is 5Sec.

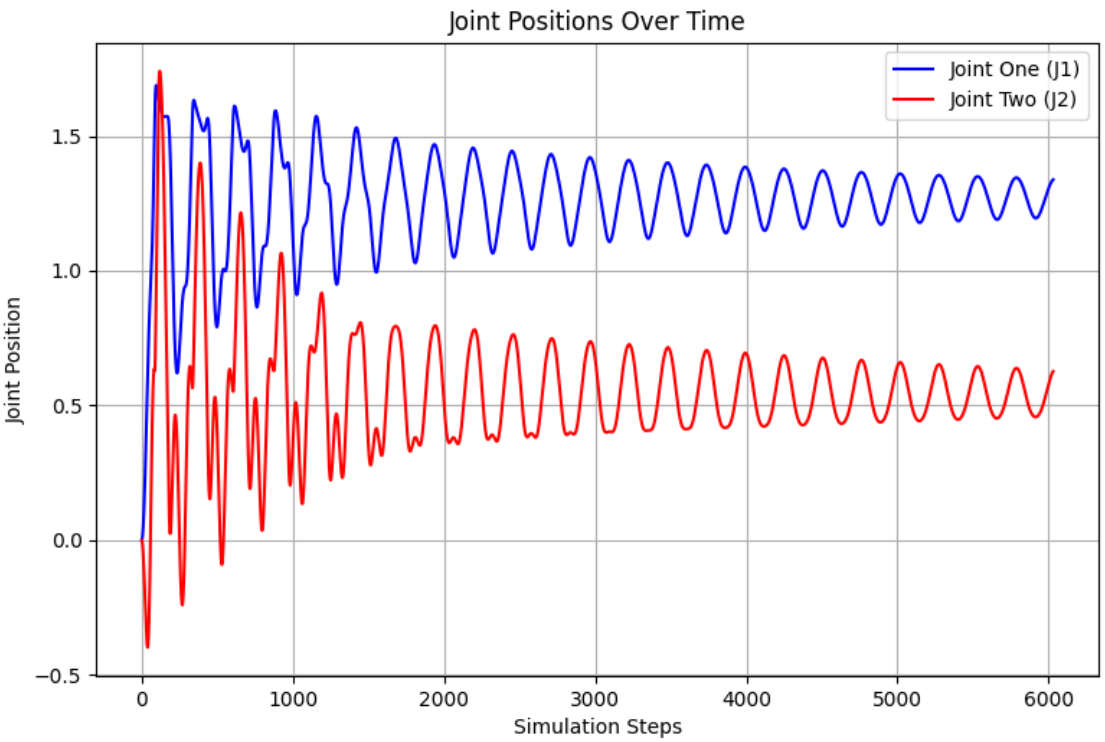

Figure 12: Value of the generalized coordinate J1 and J2 when the simulation time is 15Sec.

Analysis of the Simulation Graphs for the Passive RR Mechanism

- Both joint positions exhibit significant oscillations at the beginning of the simulation, which gradually decay over time due to damping effects.
- The elastic tendons provide restoring forces that help stabilize the joints to near-equilibrium positions.
- The J2 oscillations exhibit a slightly smaller amplitude than J1, indicating variations in tendon stiffness or its interaction with the second joint.
- The dynamic behavior suggests that the system responds dynamically to external forces and stabilizes without overshooting significantly.

Analysis of Tendon Stiffness and Damping Effects on System Dynamics

- The stiffness of the tendons and their interaction with the joints introduce oscillatory behavior, as seen in the simulation graphs for joint positions J1 (Joint One) and J2 (Joint Two).
- Damping elements in the joints help reduce oscillations over time, leading to stabilization in the system.
- The plots indicate a damped oscillatory motion in both joints, which is expected for systems with elastic cables applying restoring forces.

4 CONCLUSION

To sum up, this task successfully demonstrated the simulation of a passive RR mechanism with two elastic tendons passing over pulleys, as represented by the provided XML and Python code. The model's behavior within the simulated environment was thoroughly analyzed, and graphs of the generalized coordinates were generated. These graphs revealed that, in the presence of damping components, the system eventually reaches an equilibrium position. The XML-based simulation effectively captures the dynamics of a tendon-driven RR mechanism, showcasing the tendons' role in stabilizing joint motion. The results highlight the capability of MuJoCo to simulate complex kinematic and dynamic systems. Future work could focus on fine-tuning tendon stiffness and damping coefficients to optimize motion characteristics and system stability.